

Especificación de la clase `Order`

Arquitectura del sistema

La clase `Order` contiene los artículos comprados por un cliente de un portal de compras. Para cada pedido, se crea una `invoice`.

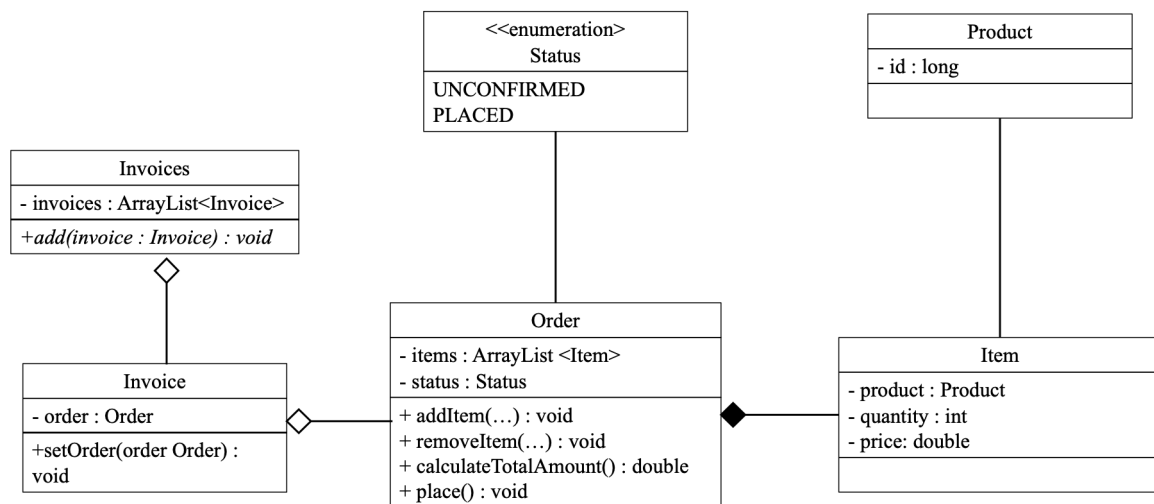


Figura 1. Diagrama de clases

La arquitectura del sistema de software se muestra en la **Figura 1**. El portal de compras vende una amplia gama de productos. Cuando un cliente quiere hacer una compra, se crea una nueva `order`. Dicha instanciación la realiza una clase controladora que no se muestra en el diagrama.

Cada vez que un cliente selecciona un `product`, se crea un nuevo `item`¹ y la clase controlador añade el `item` a la `order`. Cuando una `order` contiene `items`, adopta el `status UNCONFIRMED`; de lo contrario, el `status` será `null`. La clase controladora asegura que el `item` nunca es `null`.

El mismo `product` (con el mismo `id`) puede ser vendido por varios proveedores, y cada proveedor puede asignar un `price` distinto.

¹ Se debe tener en cuenta que la relación entre `Order` y `Product` es n:m, por lo que se requiere un objeto `Item` intermedio.

Cualquier `item` puede ser retirado del pedido. También se puede calcular el importe total de la `order`. Una vez completada la `order`, esta pasa al `status PLACED` y, a continuación, se crea una nueva `invoice` que se almacena en la clase estática `Invoices` (esta clase `Invoices` tiene como propósito evitar la creación de una base de datos dado el estado actual del proyecto).

Implementación requerida de la clase `Order`

La clase `Order` **toma los siguientes valores durante la inicialización:**

- ☐ La lista de `items` deberá estar vacía.
- ☐ El `status` de la `order` deberá tomar el valor `null`.

Se deben implementar los siguientes métodos de `Order`:

- `void addItem(Item item)`

Este método añade un `item` a la `order`

- ☐ Se debe añadir el `item` a la lista de `items`.
 - ☐ Si el `status` de la `order` es `PLACED`, se lanzará una `CannotAddItemsToPlacedOrderException`.
 - ☐ El `price` del `item` debe ser mayor o igual a cero. De lo contrario, se lanzará una `IncorrectItemException`.
 - ☐ La `quantity` del `item` debe ser mayor que cero. De lo contrario, se lanzará una `IncorrectItemException`.
 - ☐ Si el `item` ya existe en la lista de `items` y el `price` coincide, el nuevo `item` no se añadirá a la lista de `items`, sino que se incrementará la `quantity` del `item` existente.
 - ☐ Si el `item` ya existe en la lista de `items` pero el `price` es distinto, el nuevo `item` no se añadirá a la lista de `items`, sino que se incrementará la `quantity` del `item` existente. El `price` del `item` en la lista de `items` deberá ser el `price` más alto.
- ☐ Cuando se añade el primer `item` a la lista de `items`, el `status` de la `order` pasa a `UNCONFIRMED`.

- `void removeItem(Item item)`

Este método elimina un `item` de la `order`

- ☐ El `item` debe eliminarse de la lista de `items`.
- ☐ Si el `item` no existe en la lista de `items`, se deberá lanzar `NonExistingItemException`.
- ☐ Si la lista de `items` queda vacía, se le asignará el valor `null` al `status` de la `order`.